COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/18/26
NAME:	Cruz, Elijah Villaruel, Prince Benjamin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

PART 1

CODE

main.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <string>

// Function that takes an integer and a string, then prints them out
void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    // Create a thread t1 and pass it the print function with parameters
    std::thread t1(print, 10, "T.I.P.");

    // Wait for t1 to finish running before closing main
    t1.join();

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Desktop\Villaruel\HOA 13>
Printing integer: 10
Printing string: T.I.P.
```

ANALYSIS
We compiled and executed the task given that prints an integer and a string passed as arguments. We then launched a thread in the main program to run this function with sample data. Finally we called join() on the thread so our program would wait for the printing task to complete before terminating

Table 13-1. Simple One-Threaded Example

PART 2

CODE

CMakeLists.txt main.cpp

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = { "T.I.P.", "Competent", "Computer", "Engineers" };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back( x: std::thread( >> print, >> i, >> s[i]));
    }
    for (auto &th thread & : threads) {
        th.join();
    }
    return 0;
}
```

OUTPUT

```
C:\Users\Elijah\CLionProjects\SP\cmake-build-debug\SP.exe
2 : Computer1 : Competent0 : T.I.P.3 : Engineers



Process finished with exit code 0
```

ANALYSIS

We observed that the program starts four threads in a loop, and each one runs print with its own number and word. A second loop then calls join() on each thread so main waits for all of them to finish. We ran the program twice and observed different outputs. In Run 1, the lines from threads 1, 2 and 3 were mixed together on one line.
In Run 2, they came out in order. We observed this because the threads run at the same time and all write to std::cout, so we can't control which one goes first. We also observed that clean output would need a std::mutex so only one thread prints at a time. All the threads are started before any join() is called. If join() were called inside the first loop, each thread would finish before the next one started, and the program would no longer be concurrent.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Question	Villaruel	Cruz
Write a definition of multithreading and its advantages/disadvantages.	Multithreading is a programming technique where a process is split into multiple independent threads that run concurrently. Advantage: I see multithreading as a great way to boost overall app performance and CPU usage. It also uses fewer system resources than running separate full processes because threads share the same memory space. Disadvantage: I find that it adds	Multithreading is a technique where one program is split into several threads that run at the same time and share the same memory. In C++, a thread is created with std::thread. Its advantages are faster execution on multi-core CPUs, a more responsive program, and easy sharing of data between threads. Its disadvantages are data races, deadlocks, extra overhead from creating threads, and bugs that are

	extra complexity to code and makes debugging a lot harder. It also exposes the program to issues like race conditions or deadlocks if shared data is not synchronized properly.	hard to find and repeat.
Rationalize the use of multithreading by providing at least 3 use-cases.	I find multithreading essential when an application needs to manage multiple background tasks without freezing the main operations. First, I see it in web servers, where multiple incoming client requests are handled simultaneously. Second, in video game engines, separate threads, calculate physics stream audio, and render frames concurrently. Third, in in GUI desktop software, background threads handle network downloads or file reads so the main user interface stays completely inactive.	Multithreading is useful in web servers, where each client request gets its own thread so one slow request does not block the others. It is also used in GUI applications, where a background thread does heavy work such as saving or downloading so the window does not freeze. A third use is image and video processing, where each thread handles a part of the data at the same time to finish sooner.
Differentiate between parallelism and concurrency.	I view concurrency as managing multiple tasks at once by rapidly switching between them on a single core, which gives the impression that everything is running together. In contrast, I view parallelism as executing multiple tasks at the exact physical instant across separate CPU cores.	Concurrency means handling many tasks at once by overlapping them in time, even on a single core. Parallelism means running tasks at the exact same moment on multiple cores, usually on different data. In short, concurrency is about managing several tasks, while parallelism is about doing them simultaneously.

References

- cppreference.com. (n.d.). std::thread and std::mutex. https://en.cppreference.com/w/cpp/thread/thread https://en.cppreference.com/cpp/thread/mutex
- Williams, A. (2019). C++ concurrency in action (2nd ed.). Manning Publications. https://www.manning.com/books/c-plus-plus-concurrency-in-action-second-edition
- Pike, R. (2012). Concurrency is not parallelism [Waza talk]. https://go.dev/blog/waza-talk

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

CODE

CMakeLists.txt | main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>

int globalValue = 0;
std::mutex m;

void add(int value, int workMs) {
    std::this_thread::sleep_for( rtime: std::chrono::milliseconds(workMs));
    std::lock_guard<std::mutex> lock( [&] m);
    globalValue += value;
}

void display(const char *label) {
    std::lock_guard<std::mutex> lock( [&] m);
    std::cout << label << " -> globalValue = " << globalValue << std::endl;
}

int main() {
    std::thread T1( [>>] add, 10, 100);
    std::thread T2( [>>] add, 20, 200);
    std::thread T3( [>>] add, 30, 300);

    display( label: "Display (before any join)");
    T1.join();
    display( label: "Display (after T1.join())");
    T2.join();
    display( label: "Display (after T2.join())");
    T3.join();
    display( label: "Display (after T3.join())");
    return 0;
}
```

OUTPUT

```
C:\Users\Elijah\CLionProjects\SP\cmake-build-debug\SP.exe
Display (before any join) -> globalValue = 0
Display (after T1.join()) -> globalValue = 10
Display (after T2.join()) -> globalValue = 30
Display (after T3.join()) -> globalValue = 60

Process finished with exit code 0
```

ANALYSIS

Cruz

We observed that the first display showed 0 because the threads had not finished yet. After T1.join() the value was 10, after T2.join() it was 30, and after T3.join() it was 60, the sum of all three additions. This shows that join() makes the main thread wait, so the final value is only guaranteed after all three joins. Short sleeps were added to simulate work so the changes are easy to see, and a std::mutex in addition keeps the threads from changing the global variable at the same time and causing a data race.

Villaruel

We saw that the value of the global variable only changes once a thread finishes its work. The first display showed 0 because T1, T2, and T3 were all still sleeping when main printed. After T1.join() it was 10, after T2.join() it was 30, and after T3.join() it was 60. So join() makes the main wait for that specific thread, and the value builds up as each one finishes (10, then 10+ 20, then 10+20+30). The mutex keeps the updates to globalValue safe, and the different sleep times (100, 200, 300 ms) make the order of completion predictable, so we got the same output pattern we expected.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

CODE

```cpp
#include <chrono>
#include <functional>
#include <iostream>
#include <thread>
#include <vector>

// merge two sorted halves a[lo..mid] and a[mid+1..hi]
void merge(std::vector<int> &a, int lo, int mid, int hi) {
    std::vector<int> tmp;
    int i = lo, j = mid + 1;
    while (i <= mid && j <= hi) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= mid) tmp.push_back(a[i++]);
    while (j <= hi)  tmp.push_back(a[j++]);
    for (int k = 0; k < tmp.size(); k++) a[lo + k] = tmp[k];
}

void mergeSort(std::vector<int> &a, int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort( &a, lo,  hi: mid);
    mergeSort( &a,  lo: mid + 1, hi);
    merge( &a, lo, mid, hi);
}
```

```cpp
int main() {
    const int N = 2000000;
    std::vector<int> a(N);
    for (int &x : a) x = rand() % 1000000;
    std::vector<int> b = a;   // same data for both tests

    // 1) One thread
    auto start = std::chrono::steady_clock::now();
    mergeSort(a, 0, N - 1);
    auto end = std::chrono::steady_clock::now();
    std::cout << "One thread  : "
              << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
              << " ms" << std::endl;

    // 2) Two threads: each sorts one half, then we merge the halves
    start = std::chrono::steady_clock::now();
    int mid = (N - 1) / 2;
    std::thread t1(mergeSort, std::ref(b), 0, mid);
    std::thread t2(mergeSort, std::ref(b), mid + 1, N - 1);
    t1.join();
    t2.join();
    merge(b, 0, mid, N - 1);
    end = std::chrono::steady_clock::now();
    std::cout << "Two threads : "
              << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
              << " ms" << std::endl;

    std::cout << "Same result : " << (a == b ? "yes" : "no") << std::endl;
    return 0;
}
```

OUTPUT

```
C:\Users\Elijah\CLionProjects\SP\cmake-build-debug\SP.exe
One thread  : 1976 ms
Two threads : 1062 ms
Same result : yes

Process finished with exit code 0
```

```
PS C:\Users\Administrator> cd
One thread  : 1916 ms
Two threads : 927 ms
Same result : yes
```

ANALYSIS

Cruz

We observed that both methods produced the same sorted result. The two-thread version was faster (1062 ms vs 1976 ms, about 1.9 times) because each thread sorted half of the array at the same time on a different core. It was not exactly 2 times faster because the final merge is done by one thread and creating threads adds some overhead.

Villaruel

We split merge sort across two threads, one per half, then merged on main. WIth 2,000,000 integers one thread took 1916 ms and two took 927 ms, about 2.07x faster. That is slightly above the ideal 2x, which is a bit unusual. It's probably from run-to-run timing noise or cache effects, since each thread works on a smaller chunk. The final merge is still single-threaded, so we wouldn't expect it to go much past 2x. Both runs sorted correctly, so threading only changed the speed, not the output.

CONCLUSION

Cruz

In this activity, we learned how to make a C++ program do many tasks at the same time using threads. We learned that concurrency means tasks overlap and can share data, while parallelism means tasks run at the same time on different cores, usually on different data. We used std::thread to start a thread and join() to make the main thread wait for it. In Part 1, we saw the basic steps of a thread: it starts, runs its function, and is joined. In Part 2, we saw that the output was not the same every run, and in one run the lines were mixed together, so we can't tell which thread will go first when many threads print at once. In Part B, we saw that the global variable only reached 60 after all three threads were joined, and that we need a std::mutex so the threads don't change the shared variable at the same time and lose an update. In Part C, two threads sorted the data in 1062 ms while one thread took 1976 ms, so splitting the work between threads made the program faster. This also showed us that threads help the most when the work can be divided into parts that don't depend on each other. We think we did well in understanding the basics of threads, join() and mutexes, but we still need more practice with parallel algorithms.

Villaruel

This activity showed me that threads run at the same time, but I don't get to decide which one finishes first, so shared output and data need a std::mutex . In the procedure, the four-thread program printed mixed lines in one run and clean lines in the other, and join() kept main waiting until the threads finished. In the supplementary activity, join() made globalValue go 0, 10, 30, then 60, and splitting merge sort across two threads cut the time

from 1916 ms to 927 ms with the same result. I think I did, since every program ran and the outputs matched what I expected. I still need practice deciding when threading is worth it and catching data races before they show up.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction, code assistance, explanation of concepts*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: *grammar correction, code assistance, explanation of concepts*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.